Scout
Normalisation & Correlation

How Scout transforms raw data into actionable security findings

Business Overview · April 2026

Contents

What is Normalisation? 3

1.1 Software Identity Normalisation 3
1.2 CPE Normalisation 4
1.3 Vulnerability Source Normalisation 5
1.4 Version Normalisation 5

What is Correlation? 6

2.1 Stage 1 – Candidate Identification 6
2.2 Stage 2 – Version Applicability Check 7
2.3 Stage 3 – VEX Override 7
2.4 Stage 4 – Confidence Scoring 8
2.5 Stage 5 – Precedence Resolution 8

End-to-End Example: Log4Shell on SERVER-042 9

PART 1

What is Normalisation?

Scout ingests data from many different systems — ServiceNow CMDB, NVD, CISA KEV, Microsoft and Red Hat security advisories, GitHub Security Advisories, and SBOM files. Every source uses its own naming conventions, version formats, and identifiers. **Normalisation is the process of cleaning, standardising, and unifying that raw data before any matching happens.** Without it, the same product would appear under dozens of different names and vulnerabilities would be silently missed.

■ Think of normalisation as a universal translator. It ensures that 'Microsoft Corporation', 'Microsoft Corp', and 'MICROSOFT' are all recognised as the same vendor before Scout tries to match them against CVEs.

1.1 Software Identity Normalisation

When ServiceNow CMDB sends software records, publisher names and product names are inconsistent across assets. Scout normalises them into a canonical slug form and applies a vendor alias dictionary to collapse known variants.

Raw value from CMDB	After normalisation	What changed
Microsoft Corporation	microsoft	Legal suffix stripped, alias applied
Microsoft Corp	microsoft	Alias applied
Red Hat, Inc.	redhat	Punctuation removed, alias applied
Oracle America, Inc.	oracle	Legal entity collapsed to brand
Google LLC	google	Legal suffix stripped, alias applied
Microsoft Edge 120.0.2210.91 (x64) → product name	edge	Version, arch, and vendor prefix removed
Microsoft Office Professional Plus 2019 (en-US) → product	office_professional_plus	Year, locale, edition removed; stop words dropped
version string: 20231005	2023.10.05	Date-style version reformatted to dotted form

■ Stop words like 'Corporation', 'Enterprise', 'Professional', 'x64', 'en-US' are stripped from product names. This prevents the same software from being stored as multiple distinct identities just because the edition or language suffix differs.

1.2 CPE Normalisation

CPE (Common Platform Enumeration) is the industry-standard identifier used by NVD to describe which software a CVE affects. CPEs arrive in mixed case and inconsistent formats. Scout normalises every CPE to a canonical lowercase form, then builds a lookup key from vendor and product (plus optional platform

qualifiers). This key is what Scout uses to join your asset inventory against the CVE database.

Raw CPE from NVD	Normalised lookup key
cpe:2.3:a:Microsoft:Edge:120.0:*:*:*:*:*:*:*	a|microsoft|edge
CPE:2.3:A:MICROSOFT:EDGE:*:*:*:*:*:*:*:*	a|microsoft|edge (same key)
cpe:2.3:a:apache:log4j:2.14.1:*:*:*:*:*:*:*	a|apache|log4j
cpe:2.3:o:redhat:enterprise_linux:8.0:*:*:*:*:*:*:*	o|redhat|enterprise_linux

■ The CPE spec distinguishes between ANY (*) and NOT APPLICABLE (-) for each attribute. Scout preserves this distinction — collapsing them would cause false identity matches when vendor or platform qualifiers differ between products.

1.3 Vulnerability Source Normalisation

Vulnerability intelligence arrives from multiple feeds, each using its own naming. Scout maps all source names to canonical identifiers and assigns a trust priority. This priority is critical — it determines which source wins when sources disagree about whether a CVE applies to your software.

Source says	Canonical name	Trust priority	Notes
nist-nvd, NVD	nvd	100	Baseline vulnerability database
msrc-csaf, Microsoft CSAF	csaf-microsoft	90	Vendor-authoritative
redhat-csaf, Red Hat CSAF	csaf-redhat	90	Vendor-authoritative
msrc-vex, Microsoft VEX	vex-microsoft	88	Vendor exploit status
github-advisory, GHSA	ghsa	86	Package ecosystem advisories
cisa-kev, cisa kev, kve	kev	70	Exploitability signal only — not an applicability authority

CVSS scores are also normalised into consistent severity bands: ≥ 9.0 → **CRITICAL**, ≥ 7.0 → **HIGH**, ≥ 4.0 → **MEDIUM**, below 4.0 → **LOW**. This ensures the same label is shown regardless of which source originally scored the vulnerability.

1.4 Version Normalisation

A CVE typically describes affected software as a version range — for example, "Log4j 2.0 through 2.14.1". Your CMDB may report versions in many formats depending on the operating system and package manager. Scout normalises versions according to the correct scheme before comparing them.

Ecosystem / OS	Raw version from CMDB	Normalised form	Scheme used
Java / Maven	v2.13.0	2.13.0	SemVer / Maven
Python	1.0b2.post345.dev456	1.0b2.post345.dev456	PEP 440

Ecosystem / OS	Raw version from CMDB	Normalised form	Scheme used
Debian Linux	2.13.0-debian-1+deb11u1	2.13.0 (revision stripped)	DPKG
Red Hat Linux	2.13.0-1.el8	2.13.0 (RPM release stripped)	RPM
Windows (date)	20231005	2023.10.05	Date normalisation

PART 2

What is Correlation?

Normalisation cleaned and standardised the data. **Correlation is the engine that answers the real business question: Is this specific software, on this specific asset, actually vulnerable to this CVE?**

It is not a simple lookup. It is a five-stage pipeline that produces a finding with a clear verdict, a confidence score, and a full audit trail explaining every decision.

> ■ Scout does not use active scanning. It correlates what it already knows — your asset inventory from ServiceNow, and vulnerability intelligence from NVD, CSAF, VEX, and KEV — to determine exposure. Every finding is traceable back to specific evidence.

Stage 1 — Candidate Identification

For each piece of software on your assets, Scout gathers every CVE target that *could* apply. It runs four parallel lookups simultaneously, using different identifier types for breadth:

Lookup method	What it matches on	Example key
CPE index	Normalised vendor/product key	microsoft|edge
PURL index	Exact package URL from SBOM	pkg:npm/%40angular/core@15.0.0
Coord key	Ecosystem + package name	maven:org.apache.logging.log4j:log4j-core
Advisory package	GHSA / custom advisory package rules	GitHub advisory ID → log4j-core in Maven

This stage produces a *candidate list* — not confirmed findings, just potential matches that pass to the next stage for validation.

Stage 2 — Version Applicability Check

For each candidate, Scout checks whether the installed version actually falls within the vulnerable range published by the CVE. This eliminates assets that have already been patched to a safe version.

Scenario	Installed version	CVE range	Decision	Reason code
Vulnerable	2.13.0	>= 2.0, < 2.15.0	AFFECTED	within_constraints
Patched	2.15.1	>= 2.0, < 2.15.0	NOT AFFECTED	at_or_above_fixed
No version recorded	(blank)	>= 2.0, < 2.15.0	UNKNOWN	VERSION_UNKNOWN
Exact match required	3.2.1	exactly 3.2.1	AFFECTED	exact_version_match

■ UNKNOWN does not mean 'not vulnerable'. It means Scout cannot confirm either way. These findings are surfaced to analysts for manual review rather than being silently dropped.

Stage 3 — VEX Override (Vendor Statement)

Before the version check even runs, Scout checks whether the vendor has published a VEX (Vulnerability Exploitability eXchange) statement for this CVE. VEX is a vendor's formal declaration that their specific product build is or isn't affected — it takes precedence over the generic NVD range.

VEX status from vendor	Scout decision	Reason	Business meaning
known_not_affected	NOT AFFECTED	vex_not_affected	Vendor confirms their build is safe — finding suppressed
fixed	NOT AFFECTED	vex_fixed	Vendor has patched this in their release
under_investigation	UNKNOWN	vex_under_investigation	Vendor hasn't decided yet — analyst should monitor
known_affected	AFFECTED	vex_known_affected	Vendor confirms exposure — high confidence finding
no_patch	AFFECTED	vex_no_patch	Vendor acknowledges no fix exists — finding always raised
VEX is > 90 days old	UNKNOWN	vex_not_affected_stale	Stale vendor statements are not trusted — version check falls through

✓ This is how Scout eliminates false positives at scale. A Red Hat asset running the same Log4j version as an upstream open-source package may be safe because Red Hat backported a fix — their CSAF advisory says NOT_AFFECTED and Scout suppresses the finding automatically, without analyst intervention.

Stage 4 — Confidence Scoring

Each match is assigned a confidence score based on *how* it was found. A match via exact package hash is far more reliable than a match via a wildcard CPE. Confidence informs the analyst how much to trust a finding and gates which matches are allowed to become findings at all.

How the match was found	Base confidence	Cap	Meaning
Identity hash (exact cryptographic match)	94%	99%	Highest certainty — immutable identity
Exact package identifier (PURL)	90%	97%	Strong — exact package URL matched
Normalised name key	82%	90%	Good — vendor/product slug matched
CPE direct (specific version CPE)	72%	90%	Standard NVD CPE match

How the match was found	Base confidence	Cap	Meaning
Advisory package (GHSA rule)	60%	80%	Package ecosystem advisory
Coord key (ecosystem + name)	58%	78%	Moderate — ecosystem package match
CPE fallback (wildcard CPE)	54%	76%	Lower certainty — broad CPE

Adjustments applied on top of base score: **+6%** if CVE specifies an exact version, **+4%** if CVE specifies a version range, –**5%** penalty for wildcard CPE fallback. Non-CPE matches with final confidence **below 68%** are discarded — they are too uncertain to create a finding.

Stage 5 — Precedence Resolution

The same CVE can have opinions from multiple sources — NVD, Microsoft CSAF, Red Hat CSAF, and GitHub Advisories may all have a view. Scout resolves conflicts using a documented priority order, working through each tier until it finds a decisive answer.

Priority tier	Source	Behaviour when NOT AFFECTED
1 — Highest	Vendor CSAF/VEX (Microsoft, Red Hat)	Overrides all other sources — finding is suppressed
2 — High	GitHub Security Advisories (GHSA)	Wins if no vendor advisory exists
3 — Standard	NVD	Used when no vendor advisory is present
— Excluded	CISA KEV	Never used for applicability — used only to flag urgency

■ KEV (CISA Known Exploited Vulnerabilities) is intentionally excluded from applicability decisions. Being 'known exploited in the wild' means attackers are using a CVE somewhere — it does not mean your specific patched version is affected. Including KEV as an applicability authority would create false positives on patched assets. KEV is used as a prioritisation signal only — it raises the urgency of a finding that is already confirmed via the applicability pipeline.

PART 3

End-to-End Example

CVE-2021-44228 — Log4Shell · Severity: CRITICAL (CVSS 10.0)

Log4Shell was one of the most severe vulnerabilities ever discovered. It affects Apache Log4j, a Java logging library used in millions of applications. This example walks through exactly how Scout processes it from raw inventory data through to a confirmed, evidence-backed finding.

Starting Conditions

Data element	Raw value (as received)	After normalisation
Asset	SERVER-042 (from ServiceNow)	SERVER-042
Publisher	The Apache Software Foundation	apache
Product display name	Apache Log4j 2.13.0	log4j
Version	2.13.0	2.13.0 (SemVer — no change needed)
CVE	CVE-2021-44228	CVE-2021-44228 (uppercased, trimmed)
CVE source	nist-nvd	nvd (canonical name)
Affected range (NVD)	>= 2.0.0-alpha1, < 2.15.0	Parsed as: start=2.0.0-alpha1 (inclusive), end=2.15.0 (exclusive)
CPE in NVD	cpe:2.3:a:apache:log4j:*:*:*:*:*:*:*:*	Lookup key: a|apache|log4j

Pipeline Walkthrough

1 Candidate Identification

Scout derives the CPE lookup key for Log4j: a|apache|log4j. It queries the vulnerability target index and finds CVE-2021-44228 as a candidate — matched via CPE indexed direct. Match method: cpe-indexed-direct+version. Starting confidence: 72%.

2 VEX Check (runs first)

Scout checks whether Apache or a trusted vendor has published a VEX statement for this specific target. No VEX statement exists from Apache for this open-source build. Result: no VEX override — proceed to version check.

3 Version Applicability

Installed version: 2.13.0. CVE range: >= 2.0.0-alpha1 (inclusive), < 2.15.0 (exclusive). Check 1 — range start: compare(2.13.0, 2.0.0-alpha1) = +1 → 2.13.0 is above start ✓. Check 2 — range end: compare(2.13.0, 2.15.0) = -1 → 2.13.0 is below end ✓. Decision: AFFECTED. Reason code: within_constraints.

4 Confidence Scoring

Base: 72% (CPE direct). Boost: +4% (version range present). No penalties apply (direct CPE, not wildcard). Final confidence: 76%. This exceeds the 68% minimum threshold — finding is eligible.

5 Precedence Resolution

Only one source (NVD) has a view on this target. No vendor CSAF or GHSA advisory exists. NVD is the only tier with a candidate. Its decision is AFFECTED. Final state: AFFECTED. Primary source: nvd.

Finding Created

> **✓ SERVER-042 · Apache Log4j 2.13.0 · CVE-2021-44228**
> State: AFFECTED · Severity: CRITICAL · Confidence: 76%
> Matched by: cpe-indexed-direct+version · Source: NVD
> Reason: within_constraints — version 2.13.0 falls in range [2.0.0-alpha1, 2.15.0)

Contrast Scenario — Same CVE on a Red Hat Asset

A second asset, SERVER-099, is a Red Hat Enterprise Linux 8 server running Red Hat's build of Log4j 2.13.0. The version number is identical to SERVER-042 but the outcome is completely different.

Pipeline step	SERVER-042 (open-source)	SERVER-099 (Red Hat build)
Candidate found?	Yes — CPE key a|apache|log4j	Yes — same CPE key
VEX check	No VEX — skip	Red Hat CSAF: known_not_affected
Version check	2.13.0 in range → AFFECTED	Skipped — VEX overrides first
Confidence	76%	N/A — suppressed by VEX
Final state	AFFECTED	NOT AFFECTED
Reason	within_constraints	vex_not_affected
Finding raised?	Yes — CRITICAL finding	No — finding suppressed

> ■ Red Hat backports security patches into their supported release of Log4j without changing the upstream version number. Their CSAF advisory formally documents this. Scout respects that vendor statement — without it, SERVER-099 would generate a false positive finding that wastes analyst time.

Summary: What Each Stage Contributes

Stage	Purpose	Business value
Normalisation	Standardise raw data from all sources into a common language	Same software is recognised consistently regardless of how the source reported it

Stage	Purpose	Business value
Stage 1 — Candidates	Find every CVE that could apply to this software	No exposure is silently missed due to identifier mismatch
Stage 2 — Version check	Confirm the installed version is in the vulnerable range	Patched assets are not flagged — only genuinely exposed versions raise findings
Stage 3 — VEX override	Apply vendor statements that override the generic CVE range	False positives from vendor-patched builds are suppressed automatically
Stage 4 — Confidence	Score how certain each match is	Low-quality matches below the threshold never become findings — noise is reduced
Stage 5 — Precedence	Resolve conflicts when sources disagree	The most authoritative source always wins — findings reflect the best available evidence